TRAVAUX PRATIQUES · RÉSEAUX SANS FIL · M1 IBAM / UJKZ

Guide complet du TP Clustering dans les RCSF

LEACH sous **NS3** dans une VM Linux ARM (Mac Apple Silicon) : installation, concepts, code, expériences et rapport

Étudiant	KOURA Lémiyi Stéphane Ulrich (SIE)
Algorithme affecté	LEACH (Low-Energy Adaptive Clustering Hierarchy)
Simulateur affecté	NS3 (version 3.40)
Environnement	Mac Apple Silicon, VM Ubuntu 22.04 ARM64 lancée par Vagrant + QEMU
Comparaison croisée avec	Un étudiant DEEC ; sur NS3 : BOUGOUMA Leïla, sankara abdoul-guelilou, KERE Wend-panga Georges Fabrice
Livrable	Rapport de 4 pages max : courbes, critique de NS3, comparaison LEACH vs DEEC

Comment lire ce guide
Ce document explique **ce qu'il faut installer**, **ce qu'il faut faire** étape par étape jusqu'au rapport final, et **pourquoi** (les concepts derrière chaque étape). Le projet fourni contient un **Vagrantfile**, le code NS3 (`leach.cc`) et les scripts de lancement et de tracé. Ce code a été compilé et exécuté sous NS3 3.40 (build Linux x86) avant livraison ; la VM ARM elle-même n'a pas pu être testée ici.

Sommaire

1	Comprendre le sujet : ce qui est demandé
2	Les concepts à maîtriser (RCSF, clustering, LEACH, modèle radio, métriques, NS3)
3	Paramètres de simulation à fixer
4	Pas à pas : du Mac au rapport (installation, VM, validation, expériences, tracés)
5	Le code fourni (leach.cc) : structure, options, colonnes du CSV
6	Les trois expériences et les valeurs attendues
7	Traitement des résultats avec Python
8	Comparaison croisée LEACH vs DEEC
9	Rédaction du rapport (4 pages)
10	Calendrier, dépannage, pièges, check-list finale, glossaire

1. Comprendre le sujet

1.1 Ce que le TP demande

Chaque étudiant implémente **un** algorithme de clustering dans **un** simulateur, puis vérifie par la simulation les affirmations du tableau comparatif du cours (consommation d'énergie, durée de vie, scalabilité). Pour vous : **LEACH sur NS3**.

Élément	Exigence du sujet
Scénario	100 nœuds déployés aléatoirement sur 200 × 200 m ; station de base (BS) en (100, 250), donc **hors de la zone** ; énergie initiale 0,5 J ; 1000 rounds.
Nœuds avancés	20 % à 1 J uniquement pour SEP et DEEC. **Pas pour LEACH** (réseau homogène).
Modèle radio	Premier ordre : E_{elec} = 50 nJ/bit, ε_{fs} = 10 pJ/bit/m², ε_{mp} = 0,0013 nJ/bit/m⁴ (voir remarque sur l'unité, §2.5).
Métrique 1	Courbe de l'énergie résiduelle totale en fonction du temps (rounds).
Métrique 2	Passage à l'échelle : 50, 100, 300 nœuds ; mesurer le temps de setup des clusters (secondes simulées) *ou* le nombre de messages de la phase de clustering.
Métrique 3	Taux de livraison = paquets de données reçus par la BS / paquets de données émis par les capteurs. Les paquets de contrôle ne sont pas comptés.
Rapport	4 pages max : courbes, critique du simulateur (facilité d'implémentation, réalisme de la couche radio), comparaison croisée avec DEEC et explication de l'écart.
Message à démontrer	« LEACH est un piège » : la simulation doit montrer ses limites (élection aléatoire qui ignore l'énergie résiduelle et la distance, mort précoce des premiers nœuds).

1.2 Vue d'ensemble du travail

Le travail se décompose en cinq blocs, dans cet ordre : (1) comprendre les concepts ; (2) installer NS3 et écrire la simulation LEACH ; (3) valider sur un petit cas puis comparer à une référence ; (4) lancer les trois expériences avec plusieurs graines aléatoires ; (5) tracer, comparer avec DEEC et rédiger. Le plus long est le bloc 2, lancez-le tôt.

2. Les concepts à maîtriser

2.1 Réseau de capteurs sans fil (RCSF) et énergie

Un RCSF est un ensemble de petits nœuds autonomes (capteur, processeur, radio, batterie) qui mesurent un phénomène (température, humidité…) et envoient leurs mesures vers une **station de base** (BS ou « sink »). La batterie est généralement **non remplaçable** : la durée de vie du réseau dépend donc de l'énergie. Point clé : **communiquer coûte beaucoup plus cher que calculer**, et l'énergie d'une transmission augmente très vite avec la distance. Tout l'enjeu des protocoles de clustering est de réduire et de répartir le coût des transmissions.

2.2 Le clustering

Le réseau est découpé en groupes (**clusters**). Dans chaque cluster, un nœud est le **Cluster Head (CH)** ; les autres sont des **membres**. Les membres envoient leurs données au CH sur une courte distance (peu coûteux). Le CH **agrège** les données reçues (fusion en un seul paquet) puis transmet à la BS sur une longue distance (coûteux). Trois bénéfices : moins de longues transmissions, moins de données à envoyer grâce à l'agrégation, et la possibilité de **faire tourner** le rôle de CH pour répartir la charge.

> **À retenir**
> Le rôle de CH est très coûteux (réception de tous les membres + agrégation + transmission longue distance). Si ce rôle est mal réparti, certains nœuds meurent très tôt. C'est exactement ce que les algorithmes comparés dans le TP cherchent à améliorer, et ce que LEACH fait mal.

2.3 Notion de round

Le fonctionnement est découpé en **rounds**. Un round contient deux phases : la **phase d'initialisation (setup)** où l'on élit les CH et forme les clusters, et la **phase de régime permanent (steady state)** où les membres envoient leurs données. Le sujet demande 1000 rounds : c'est votre axe de temps pour les courbes.

2.4 L'algorithme LEACH

LEACH (Heinzelman et al., 2000) est le protocole de clustering de référence : distribué, probabiliste, pour réseau homogène.

Phase 1 : élection des CH

Au début du round *r*, chaque nœud *n* qui n'a **pas encore été CH dans l'époque courante** (1/p rounds ; l'ensemble G de LEACH, remis à zéro au début de chaque époque) tire un nombre aléatoire uniforme entre 0 et 1. Il devient CH si ce nombre est inférieur au seuil :

$$T(n) = p \,/\, \left(1 - p \cdot (r \bmod 1/p) \right)$$

p est la proportion souhaitée de CH (typiquement 0,05, soit en moyenne 5 CH pour 100 nœuds). L'idée : sur une « époque » de 1/p = 20 rounds, chaque nœud doit être CH une fois. Au début de l'époque (r mod 20 = 0), T = p = 0,05. À la fin (r mod 20 = 19), T = 1 : tous les nœuds qui n'ont pas encore été CH le deviennent avec certitude. Les nœuds ayant déjà été CH dans l'époque ont T = 0. **Attention à l'implémentation** : si l'on interdit simplement « d'être CH pendant les 20 derniers rounds » au lieu de remettre G à zéro à chaque époque, les nœuds se désynchronisent et des rounds à 40 CH apparaissent. Le code fourni remet G à zéro à chaque époque.

Phase 2 : formation des clusters

1. Chaque CH diffuse un message d'annonce (**ADV**).
2. Chaque nœud non-CH écoute les annonces et choisit le CH dont le signal est le plus fort, c'est-à-dire en pratique le **plus proche**, puis lui envoie un message d'adhésion (**JOIN**).
3. Chaque CH construit un planning TDMA (un créneau par membre) et le diffuse (**SCHEDULE**). Cela évite les collisions entre membres et permet à leur radio de dormir hors de leur créneau.

Phase 3 : régime permanent

Chaque membre envoie son paquet de données au CH pendant son créneau. Le CH reçoit tout, agrège, puis envoie **un seul paquet** à la BS. Le round suivant recommence par l'élection.

Pourquoi LEACH est « un piège »

- **Il ignore l'énergie résiduelle** : un nœud presque vide peut être élu CH et mourir.
- **Il ignore la position** : un CH loin de la BS paie la transmission en d^4 (très coûteuse) ; un CH mal placé donne des clusters déséquilibrés.
- **Le nombre de CH varie** d'un round à l'autre (il n'est que de p·N en moyenne) ; il peut même être nul, auquel cas il faut définir un comportement de repli (envoi direct à la BS, très coûteux).
- **Résultat attendu** : le premier nœud meurt tôt (FND précoce) alors que le réseau a encore beaucoup d'énergie au total (dans mon essai : environ 58 % de l'énergie du réseau au moment du FND) ; c'est la limite à mettre en évidence par la simulation.

2.5 Le modèle radio de premier ordre

C'est le modèle d'énergie imposé. Pour envoyer *k* bits sur une distance *d* :

$$E_{Tx}(k,d) = E_{elec} \cdot k + \varepsilon_{fs} \cdot k \cdot d^2 \text{ si } d < d_0 \text{ (espace libre)}$$

$$E_{Tx}(k,d) = E_{elec} \cdot k + \varepsilon_{mp} \cdot k \cdot d^4 \text{ si } d \geq d_0 \text{ (multi-trajets)}$$

$$E_{Rx}(k) = E_{elec} \cdot k \quad E_{DA} = 5 \text{ nJ/bit/signal (agrégation)}$$

E_{elec} est l'énergie de l'électronique (émetteur ou récepteur), indépendante de la distance. Le terme d'amplification dépend de la distance : en **d^2** proche, en **d^4** loin. Le seuil **d_0** est la distance où les deux termes sont égaux : $\varepsilon_{fs} \cdot d^2 = \varepsilon_{mp} \cdot d^4$, donc $d_0 = \sqrt{(\varepsilon_{fs} / \varepsilon_{mp})}$.

> **Attention : unité de ε_{mp}**
> L'énoncé écrit ε_{mp} = 0,0013 **nJ**/bit/m⁴. Dans la littérature LEACH, la valeur est 0,0013 **pJ**/bit/m⁴, ce qui donne $d_0 \approx 87{,}7$ m, cohérent avec une zone de 200 m. Avec des nJ, d_0 serait $\sqrt{(10 \text{ pJ} / 1{,}3 \text{ nJ})} \approx 2{,}8$ m, et presque toutes les transmissions seraient en d⁴ avec un coût 1000 fois trop élevé. **Recommandation** : utilisez les pJ, indiquez-le clairement dans le rapport, et si possible faites confirmer par l'enseignant. Le sujet autorise de justifier ses valeurs.

Exemple de calcul à refaire à la main (paquet de 4000 bits)

Transmission	Calcul	Énergie
Membre → CH à 30 m ($d < d_0$)	50 nJ × 4000 + 10 pJ × 4000 × 30^2	200 µJ + 36 µJ = **236 µJ**
CH → BS à 150 m ($d \geq d_0$)	200 µJ + 0,0013 pJ × 4000 × 150^4	200 µJ + 2,63 mJ = **2,83 mJ**
CH → BS à 200 m	200 µJ + 0,0013 pJ × 4000 × 200^4	≈ **8,5 mJ**
CH → BS à 270 m (coin opposé)	200 µJ + 0,0013 pJ × 4000 × 270^4	≈ **27,8 mJ**
Réception d'un paquet de 4000 bits	50 nJ × 4000	**200 µJ**

Une transmission vers la BS depuis le coin le plus éloigné coûte donc environ **120 fois** une transmission membre → CH. La BS étant à (100, 250), les distances vont d'environ 50 m à 270 m : la quasi-totalité des CH sont en régime d⁴. C'est ce qui rend ce scénario très défavorable à LEACH et très instructif pour votre comparaison.

2.6 Les métriques

Métrique	Définition	Intérêt
Énergie résiduelle totale	Somme des énergies restantes de tous les nœuds, relevée à chaque round.	Vitesse de consommation du réseau.
FND (First Node Dies)	Round où le premier nœud meurt (énergie = 0).	Période de stabilité ; point faible de LEACH.
HND (Half Nodes Die)	Round où 50 % des nœuds sont morts.	Durée de vie « médiane ».
LND (Last Node Dies)	Round où le dernier nœud meurt. Peut ne pas être atteint en 1000 rounds : notez alors « > 1000 ».	Durée de vie maximale.
Taux de livraison	Paquets de données reçus par la BS / paquets de données émis par les capteurs.	Fiabilité de la collecte. Exclut les paquets de contrôle.
Temps de setup	Durée simulée (s) entre le début du round et la fin de la formation des clusters.	Passage à l'échelle.
Messages de clustering	Nombre de messages ADV + JOIN + SCHEDULE par round.	Alternative au temps de setup, plus simple à mesurer.

2.7 Les concepts NS3 utiles

Concept NS3	À quoi ça sert dans ce TP
`Node`, `NodeContainer`	Représentent les capteurs. `NodeContainer nodes; nodes.Create(100);`
`MobilityHelper`, `ConstantPositionMobilityModel`, `RandomRectanglePositionAllocator`	Donnent à chaque nœud une position fixe aléatoire dans 200 × 200 m. On lit la position avec `GetObject<MobilityModel>()->GetPosition()`.

Concept NS3	À quoi ça sert dans ce TP
`Simulator::Schedule`, `Simulator::Run/Stop/Destroy`	Planification d'événements (début de chaque round) et exécution de la simulation. Le temps simulé est indépendant du temps réel.
`RngSeedManager::SetSeed / SetRun`, `UniformRandomVariable`	Générateur aléatoire reproductible. Changer `SetRun` donne une nouvelle « graine » : indispensable pour répéter 5 fois chaque scénario.
`CommandLine`	Passer n, graine, p, fichier de sortie au programme sans recompiler.
`scratch/`	Dossier où l'on place un programme C++ perso ; il est compilé automatiquement par `./ns3 build`.
Cadre d'énergie NS3 (`EnergySource`, `WifiRadioEnergyModel`)	Existe, mais calibré sur du Wi-Fi/802.15.4 réel, pas sur le modèle de premier ordre. **On écrit son propre modèle d'énergie** (quelques lignes) pour utiliser exactement les valeurs du sujet.

3. Paramètres de simulation à fixer

Les paramètres imposés ne suffisent pas. Les autres doivent être choisis, **documentés dans un tableau du rapport** et **identiques à ceux de votre binôme DEEC** (mettez-vous d'accord dès le début).

Paramètre	Valeur	Origine / justification
Nombre de nœuds N	100 (puis 50 et 300)	Imposé
Zone	200 × 200 m, positions aléatoires	Imposé
Station de base	(100, 250)	Imposé
Énergie initiale	0,5 J pour tous	Imposé (LEACH est homogène)
Rounds	1000	Imposé
E_{elec}, ε_{fs}, ε_{mp}	50 nJ/bit ; 10 pJ/bit/m² ; 0,0013 pJ/bit/m⁴	Imposé, avec correction d'unité à justifier
d_0	≈ 87,7 m	Calculé
E_{DA}	5 nJ/bit/signal	Valeur usuelle de la littérature
Taille paquet de données	4000 bits	Valeur usuelle LEACH ; choix libre à justifier
Taille paquet de contrôle	200 bits	Choix libre
p (proportion de CH)	0,05	Valeur de référence de LEACH
Débit radio (pour le temps de setup)	250 kbit/s	Débit IEEE 802.15.4 ; choix libre
Portée de l'ADV	100 m (ou toute la zone)	Choix libre : coût d'énergie de l'ADV
Nombre de graines	5 minimum (idéalement 10)	Fiabilité statistique

Ordre de grandeur à connaître
Avec 4000 bits et la BS à (100, 250), mon essai donne (5 graines, NS3) : premier nœud mort vers le round **190**, moitié des nœuds morts vers **520**, tous morts vers **850**. Avec 2000 bits, le FND passe à environ 400 ; avec 500 bits, aucun nœud ne meurt en 1000 rounds. La taille du paquet change donc fortement les courbes : fixez-la une fois, justifiez-la, et prenez **la même que votre binôme DEEC**.

4. Pas à pas : du Mac au rapport

Cette partie est la feuille de route complète. Les commandes sont à taper dans le Terminal du Mac (« Mac ») ou dans la VM après `vagrant ssh` (« VM »).

Étape 0 · Préparer le Mac (une seule fois, ≈ 15 min)

Pourquoi QEMU et pas VirtualBox ? Un Mac Apple Silicon a un processeur ARM. VirtualBox ne sait pas y faire tourner une VM Ubuntu x86, et les boîtes Vagrant classiques (`ubuntu/jammy64`) sont en x86. On utilise donc une boîte Ubuntu **ARM64** et le provider **QEMU**, qui profite de l'accélération matérielle d'Apple (HVF). NS3 se compile très bien sur ARM64.

```
# Mac : vérifier que c'est bien une puce Apple (M1, M2, M3, M4)
uname -m                      # doit afficher : arm64

# Homebrew (si absent) : https://brew.sh
brew install qemu
brew tap hashicorp/tap && brew install hashicorp/tap/hashicorp-vagrant
vagrant plugin install vagrant-qemu

# vérifications
vagrant --version
qemu-system-aarch64 --version
vagrant plugin list           # doit contenir vagrant-qemu
```

Ressources
Prévoyez au moins **10 Go d'espace libre** et **8 Go de RAM** sur le Mac (la VM prend 4 Go par défaut). Fermez les applications gourmandes pendant la compilation de NS3.

Étape 1 · Récupérer le projet fourni

Dézippez le dossier du projet (par exemple `~/rcsf-ns3-arm`). Il contient :

Fichier	Rôle
`Vagrantfile`	Décrit la VM (Ubuntu 22.04 ARM64, QEMU), installe les paquets, copie `tp_rcsf/` dans la VM, installe et compile NS3, teste leach.cc.
`tp_rcsf/leach.cc`	La simulation LEACH sous NS3 (modèle radio de premier ordre).
`tp_rcsf/scripts/install_ns3.sh`	Installe NS3 3.40 dans `~/ns3` (appelé par le Vagrantfile, relançable à la main).
`tp_rcsf/scripts/run_all.sh`	Lance toutes les expériences (A, B, C) avec 5 graines et écrit les CSV.
`tp_rcsf/scripts/plot_results.py`	Calcule FND/HND/LND, taux de livraison, passage à l'échelle ; trace les figures ; compare avec DEEC.
`tp_rcsf/scripts/ref_leach.py`	Simulation Python de référence pour vérifier l'ordre de grandeur de vos résultats.
`GUIDE_TP_RCSF.pdf`	Ce guide.

Étape 2 · Démarrer la VM (≈ 15 à 40 min la première fois)

```
# Mac, dans le dossier du projet
cd ~/rcsf-ns3-arm
vagrant up --provider=qemu       # --provider=qemu est obligatoire à chaque "vagrant up"
vagrant ssh                      # entre dans la VM
```

Ce que fait `vagrant up` : téléchargement de la boîte Ubuntu ARM64 (quelques centaines de Mo), démarrage de la VM, installation des paquets (g++, cmake, Python, pandas, matplotlib), création d'un swap de 2 Go, copie des fichiers du TP, clonage de NS3 3.40 (profil optimisé, modules *core, network, mobility* seulement pour aller vite), compilation, puis un test de 5 rounds de leach.cc.

Bon à savoir
Si l'installation est interrompue (coupure, fermeture du Terminal), relancez simplement `vagrant provision` : les étapes déjà faites sont sautées. Pour changer les ressources : `VM_MEMORY=6G VM_CPUS=4 vagrant up --provider=qemu`.

Commandes Vagrant à connaître

Commande (Mac)	Effet
`vagrant ssh`	Se connecter à la VM
`vagrant halt`	Éteindre la VM (les fichiers sont conservés)
`vagrant up --provider=qemu`	Rallumer la VM

Commande (Mac)	Effet
`vagrant provision`	Rejouer l'installation (relance NS3 si besoin)
`vagrant destroy`	Supprimer la VM (**efface aussi vos résultats dans la VM** : copiez-les d'abord)

Étape 3 · Vérifier l'installation (VM)

```
ls ~/ns3 ~/tp_rcsf                                  # NS3 et les fichiers du TP sont là
cd ~/ns3
./ns3 run --no-build "scratch/leach --n=20 --rounds=5 --out=/tmp/test.csv"
head -n 3 /tmp/test.csv                             # le CSV doit avoir des colonnes et des lignes
python3 ~/tp_rcsf/scripts/ref_leach.py              # référence Python (quelques secondes)
```

Si la première commande affiche une ligne « FND=… HND=… PDR=… » sans erreur, NS3 et leach.cc fonctionnent.

Étape 4 · Comprendre le code avant de lancer

Lisez `~/tp_rcsf/leach.cc` en suivant le §5 de ce guide : constantes radio, fonctions TxE/RxE, puis les quatre étapes d'un round (`ElectCH`, `FormClusters`, `SteadyState`, `Record`). Vous devez pouvoir expliquer chaque ligne au rapport et en soutenance.

Étape 5 · Valider à la main (10 nœuds)

1. Lancez un petit scénario : `./ns3 run --no-build "scratch/leach --n=10 --rounds=3 --out=/tmp/t10.csv"`.
2. Avec 10 nœuds à 0,5 J, l'énergie initiale totale est 5 J. Ouvrez le CSV : `total_energy` au round 0 doit être un peu inférieur à 5 J. La différence est la consommation du round.
3. Refaites un calcul à la main avec les formules du §2.5 (par exemple un membre à 30 m : 236 µJ ; un CH à 150 m de la BS : 2,83 mJ) et vérifiez que l'ordre de grandeur colle avec la consommation observée.
4. Vérifiez la reproductibilité : deux exécutions avec la même graine (`--run=1`) donnent exactement le même CSV ; `--run=2` donne autre chose.
5. Vérifiez le nombre de CH : sur 100 nœuds, la moyenne de `nCH` doit être proche de 5 (le script de tracé l'affiche).

Étape 6 · Lancer toutes les expériences (VM, ≈ 1 min)

```
bash ~/tp_rcsf/scripts/run_all.sh          # 5 graines par scénario (ajouter un nombre pour en changer : ... run_all.sh 10)
ls ~/tp_rcsf/results                       # leach_n100_run1.csv ... leach_n50_..., leach_n300_..., leach_per_n100_...
```

Le script exécute : 100 nœuds (expériences A et C), 50 et 300 nœuds (expérience B), et 100 nœuds avec 2 % de pertes (variante du taux de livraison). Chaque exécution affiche FND, HND, LND et le taux de livraison.

Étape 7 · Tracer et obtenir les chiffres du rapport (VM)

```
python3 ~/tp_rcsf/scripts/plot_results.py
cat ~/tp_rcsf/figures/resume.txt           # FND/HND/LND, taux de livraison, échelle : à recopier dans le rapport
ls ~/tp_rcsf/figures                       # les figures PNG
```

Étape 8 · Récupérer les figures sur le Mac

```
# Mac, dans le dossier du projet
vagrant ssh-config > ssh.cfg
scp -F ssh.cfg -r default:~/tp_rcsf/figures ./figures
scp -F ssh.cfg -r default:~/tp_rcsf/results ./results
```

Pour envoyer un fichier modifié vers la VM (par exemple un `leach.cc` édité sur le Mac) : `scp -F ssh.cfg leach.cc default:~/tp_rcsf/leach.cc`, puis relancer `run_all.sh` (il recopie leach.cc dans NS3 et recompile).

Étape 9 · Comparaison avec DEEC

Voir le §8. Concrètement : récupérez les CSV DEEC de votre binôme (même format), copiez-les dans la VM (par exemple ~/deec/), puis :

```
python3 ~/tp_rcsf/scripts/plot_results.py --compare "~/deec/deec_n100_run*.csv" --label DEEC --other-init 60
```

Étape 10 · Rédiger le rapport

Voir le §9 : 4 pages, courbes, critique de NS3, comparaison LEACH/DEEC. Prenez les chiffres dans `figures/resume.txt` et les images dans `figures/`.

Choix de modélisation à justifier dans le rapport

Option	Principe	Avantages	Inconvénients
A : niveau applicatif (code fourni)	Nœuds et positions NS3 ; énergie par formules du modèle de premier ordre ; canal idéal, avec option de pertes aléatoires (`--per`).	Contrôle total des valeurs imposées, fidèle au papier LEACH, rapide.	Couche radio peu réaliste (pas de collisions ni d'interférences) : à dire dans la critique de NS3.
B : paquets réels (lr-wpan, Wi-Fi)	Vrais paquets NS3 sur un canal avec propagation, MAC, collisions.	Réalisme, taux de livraison plus parlant.	Beaucoup plus de code ; l'énergie ne suit pas le modèle imposé sans adaptation. Compiler les modules supplémentaires : `NS3_MODULES="" vagrant provision` (≈ 1 h).

5. Le code fourni (tp_rcsf/leach.cc)

Le programme est une simulation **par rounds** écrite au niveau applicatif, qui utilise NS3 pour les nœuds, les positions, le temps simulé et le générateur aléatoire. Il ne dépend que des modules *core*, *network* et *mobility* : la compilation de NS3 est donc rapide.

5.1 Structure du programme

Élément	Rôle
`TxE(bits, d)`, `RxE(bits)`	Modèle radio de premier ordre (§2.5) : énergie d'émission (d^2 si $d < d_0$, sinon d^4) et de réception.
`NodeContainer`, `RandomRectanglePositionAllocator`, `ConstantPositionMobilityModel`	Création des 100 nœuds et positions aléatoires fixes dans 200 × 200 m. La BS est un point (100, 250).
`ElectCH()`	Élection LEACH : seuil T(n), ensemble G remis à zéro à chaque époque (1/p rounds).
`FormClusters()`	Phase d'initialisation : ADV, JOIN (CH le plus proche), SCHEDULE ; consommation d'énergie, compteur de messages, temps de setup.
`SteadyState()`	Membres → CH, agrégation (E_{DA}), CH → BS ; compteurs de paquets de données émis/reçus. Repli : si aucun CH n'est élu, les nœuds envoient directement à la BS.
`Record()`	Écrit une ligne de CSV par round et mémorise FND, HND, LND.
`Simulator::Schedule`	Enchaîne les rounds (1 round = 1 s simulée).
`RngSeedManager::SetRun`	Graine reproductible : `--run=1..5`.

5.2 Options de la ligne de commande

Option	Défaut	Signification
`--n`	100	Nombre de nœuds
`--rounds`	1000	Nombre de rounds
`--run`	1	Numéro de graine (positions et élections aléatoires)
`--p`	0.05	Proportion de CH
`--kData`, `--kCtrl`	4000, 200	Taille des paquets de données et de contrôle (bits)
`--e0`	0.5	Énergie initiale de chaque nœud (J)
`--per`	0	Taux de perte aléatoire (0 = canal idéal)
`--area`, `--bsx`, `--bsy`	200, 100, 250	Côté de la zone ; position de la BS
`--advRange`	100	Portée (m) des messages ADV et SCHEDULE
`--out`	results/leach.csv	Fichier CSV de sortie

5.3 Colonnes du CSV (une ligne par round)

Colonne	Contenu
`round`	Numéro du round (à partir de 0)
`total_energy`	Énergie résiduelle totale du réseau (J)
`alive`	Nombre de nœuds vivants
`nCH`	Nombre de CH élus dans ce round

Colonne	Contenu
`data_sent`, `data_recv`	Paquets de données émis par les capteurs / reçus par la BS, **cumulés**. Un paquet agrégé reçu compte pour tous les paquets sources qu'il représente.
`ctrl_msgs`	Messages de contrôle (ADV + JOIN + SCHEDULE), cumulés. Jamais comptés dans le taux de livraison.
`setup_time`	Temps de setup de ce round (s simulées) : ADV des CH en séquence + JOIN du CH le plus chargé + SCHEDULE, à 250 kbit/s
`mean_energy_alive`, `mean_energy_CH`	Énergie moyenne des nœuds vivants / des CH élus du round

5.4 Le cœur de LEACH : l'élection

```
void LeachSim::ElectCH() {
  m_ch.clear();
  std::fill(m_isCH.begin(), m_isCH.end(), false);
  const int period = (int)std::round(1.0 / m_p);          // époque = 1/p rounds (20)
  const int epoch  = (int)m_r / period;
  const double T = m_p / (1.0 - m_p * (double)(m_r % period));
  for (uint32_t i = 0; i < m_n; i++) {
    if (!m_s[i].alive) continue;
    const bool eligible = m_s[i].lastEpoch < epoch;       // pas encore CH dans cette époque
    if (eligible && m_rng->GetValue() < T) {              // tirage aléatoire contre le seuil
      m_ch.push_back(i); m_isCH[i] = true; m_s[i].lastEpoch = epoch;
    }
  }
}
```

Remarquez ce qui manque : **aucune référence à l'énergie résiduelle ni à la distance à la BS**. C'est exactement la faiblesse de LEACH que le TP veut vous faire mettre en évidence.

5.4 bis Si votre version de NS3 diffère

- Le code a été compilé avec NS3 3.40. Avec une version plus récente, les noms de classes utilisés (`NodeContainer`, `MobilityHelper`, `RandomRectanglePositionAllocator`, `UniformRandomVariable`, `Simulator::Schedule`, `CommandLine`) sont stables ; une erreur de compilation vient en général d'un include ou d'un changement mineur de signature : lisez le message du compilateur.
- Pour changer la version installée : `NS3_VERSION=3.41 vagrant provision` (après suppression de `~/.ns3_ready_3.40` dans la VM).
- Si vous éditez `leach.cc`, relancez simplement `run_all.sh` : il le recopie dans `~/ns3/scratch/` et recompile.

6. Les trois expériences et les valeurs attendues

Valeurs de repère
Les valeurs ci-dessous viennent de l'exécution de `leach.cc` sous NS3 3.40 (5 graines, paramètres du §3, 4000 bits). Vos valeurs seront proches mais pas identiques si vous changez un paramètre ; si elles sont très différentes, cherchez l'erreur (unité de ε_{mp}, taille de paquet, coût du ADV).

6.1 Expérience A : énergie (100 nœuds, 1000 rounds)

Objectif : montrer comment le réseau se vide et mettre en évidence la limite de LEACH. **Commande** : `run_all.sh` (graines 1 à 5, `--n=100`) puis `plot_results.py`.

Mesure	Valeur de repère (5 graines)
FND (premier nœud mort)	≈ 191 ± 14 rounds
HND (moitié des nœuds morts)	≈ 519 ± 29 rounds
LND (dernier nœud mort)	≈ 844 ± 31 rounds
Énergie restante au moment du FND	≈ 58 % de l'énergie initiale du réseau
Énergie restante au round 1000	≈ 0 J
Nombre de CH par round	moyenne ≈ 5,0, écart-type ≈ 2,2, de 0 à 13 ; ≈ 4 rounds sur 950 sans aucun CH

Figures produites : `energie.png` (énergie résiduelle totale), `vivants.png` (nœuds vivants), `nch_histogramme.png` (variabilité du nombre de CH), `energie_ch_vs_moyenne.png` (énergie des CH élus comparée à la moyenne du réseau : les deux courbes se confondent, preuve que LEACH ne favorise pas les nœuds riches).

Lecture de la courbe
La courbe d'énergie descend presque linéairement puis s'aplatit : au début, 5 CH par round paient des transmissions en d^4 vers une BS lointaine. Le premier nœud meurt à 191 rounds alors que 58 % de l'énergie du réseau reste disponible : l'énergie est mal répartie, pas épuisée. Les nœuds morts en premier sont ceux élus CH alors qu'ils sont loin de la BS.

6.2 Expérience B : passage à l'échelle (50, 100, 300 nœuds)

Objectif : mesurer comment le coût de la phase de clustering évolue avec la taille du réseau. **Choix à écrire dans le rapport** : la surface reste 200 × 200 m, donc la densité augmente avec N.

N	Temps de setup moyen	Messages de contrôle par round	CH par round	FND
50	≈ 26 ms	≈ 48	≈ 2,6	≈ 99
100	≈ 38 ms	≈ 104	≈ 5,0	≈ 191
300	≈ 61 ms	≈ 315	≈ 14,9	≈ 271

- Les messages de clustering sont **linéaires en N** (≈ N + 2·nCH, avec nCH = p·N) : LEACH passe bien à l'échelle côté protocole.
- Le temps de setup croît moins vite que N : les JOIN sont sérialisés par CH, et plus il y a de CH, plus chacun a de membres en parallèle. Il dépend de mon modèle (250 kbit/s, ADV en séquence) : précisez vos hypothèses dans le rapport.
- Le FND augmente avec N parce que, dans la même surface, les clusters sont plus petits et les liaisons membre → CH plus courtes. Ce n'est pas une amélioration de l'algorithme : discutez-le.
- Le temps de setup est en millisecondes simulées ; l'axe des figures est en ms. Si votre binôme DEEC utilise un autre modèle de temps, comparez plutôt les **messages**.

6.3 Expérience C : taux de livraison

Objectif : mesurer la proportion de données qui atteignent la BS (paquets de données seulement).
Commande : mêmes CSV que l'expérience A, plus la variante `leach_per_n100_run*.csv` (2 % de pertes).

Mesure	Valeur de repère
Taux de livraison global (canal idéal)	≈ 98,9 % (de 98,2 % à 99,2 % selon la graine)
Taux de livraison global avec 2 % de pertes	≈ 95,5 %
Paquets livrés par round	100 au début → 0 vers le round 850

> **Piège d'interprétation**
> Avec ce modèle, le taux de livraison (reçus / émis) reste proche de 100 % **même quand le réseau s'effondre**, parce que les nœuds morts n'émettent plus et ne comptent donc pas au dénominateur. La perte vient seulement des CH qui meurent en plein round ou des pertes aléatoires. Pour ne pas conclure à tort que « tout va bien », la figure `livraison_par_round.png` montre la quantité de données réellement livrées à la BS par round, qui s'effondre. Présentez les **deux** et expliquez la différence.

Figures produites : `livraison.png` (taux par fenêtre de 50 rounds), `livraison_par_round.png` (débit utile), `echelle.png` (expérience B). Rappel de la consigne : **seuls les paquets de données** comptent, pas ADV/JOIN/SCHEDULE.

6.4 Récapitulatif

Expérience	Scénario lancé par run_all.sh	Fichiers produits	Sorties à exploiter
A : énergie	`--n=100 --run=1..5`	`leach_n100_run*.csv`	total_energy, alive, nCH
B : échelle	`--n=50` et `--n=300` (et n=100)	`leach_n50_*`, `leach_n300_*`	setup_time, ctrl_msgs
C : livraison	n=100 (et `--per=0.02`)	`leach_n100_*`, `leach_per_n100_*`	data_sent, data_recv

7. Traitement des résultats avec Python

7.1 Pourquoi plusieurs graines

Les positions des nœuds et l'élection des CH sont aléatoires : une seule exécution peut être atypique. On répète donc chaque scénario avec plusieurs graines (`--run=1..5`) et on présente la **moyenne**, idéalement avec l'**écart-type** (les bandes ombrées des figures). C'est ce qui rend vos conclusions défendables.

7.2 Le script plot_results.py

```
python3 ~/tp_rcsf/scripts/plot_results.py                      # tout, valeurs par défaut
python3 ~/tp_rcsf/scripts/plot_results.py --results ~/tp_rcsf/results --figures ~/tp_rcsf/figures
python3 ~/tp_rcsf/scripts/plot_results.py --compare "~/deec/deec_n100_run*.csv" --label DEEC --other-init 60
```

Figure / fichier	Contenu
`energie.png`	Énergie résiduelle totale (moyenne ± écart-type)
`vivants.png`	Nœuds vivants au cours du temps
`energie_normalisee.png`	Énergie / énergie initiale, LEACH vs autre algorithme (avec `--compare`)
`nch_histogramme.png`	Distribution du nombre de CH par round
`energie_ch_vs_moyenne.png`	Énergie des CH élus vs énergie moyenne des vivants

Figure / fichier	Contenu
`livraison.png`, `livraison_par_round.png`	Taux de livraison par fenêtre ; données livrées par round
`echelle.png`	Temps de setup et messages en fonction de N
`resume.txt`	Tous les chiffres : FND/HND/LND par graine et en moyenne, livraison, échelle

Chaque figure doit avoir dans le rapport : axes avec unités, légende, nombre de graines indiqué, et la même mise en forme pour LEACH et DEEC.

7.3 Simulation Python de référence (ref_leach.py)

`ref_leach.py` est une réimplémentation indépendante et simplifiée de LEACH (mêmes formules et paramètres, autre générateur aléatoire). Elle sert à **contrôler l'ordre de grandeur** de votre simulation NS3, pas à la remplacer. Exemple : `python3 ~/tp_rcsf/scripts/ref_leach.py --k 4000 --kc 200 --seeds 10`.

Paquet de données (ctrl)	FND	HND	Énergie restante au round 1000
4000 bits (200)	≈ 206 (NS3 : ≈ 191)	≈ 526 (NS3 : ≈ 519)	≈ 0 J : tous les nœuds sont morts en 10 graines sur 10
2000 bits (200)	≈ 406	≈ 947 (atteint dans 7 graines sur 10)	≈ 3,8 J
500 bits (50)	non atteint en 1000 rounds	non atteint	≈ 35,7 J sur 50 J

Un accord à quelques pourcents près entre NS3 et la référence Python est normal. Un écart d'un facteur 2 ou plus doit vous alerter : unité de ε_{mp}, énergie de réception du CH oubliée, repli sans CH, ou règle d'éligibilité (G non remis à zéro à chaque époque, qui provoque des rounds à plus de 40 CH).

8. Comparaison croisée LEACH vs DEEC

8.1 Organisation

- Contactez votre binôme DEEC (idéalement l'un des trois étudiants DEEC sur NS3 : BOUGOUMA Leïla, sankara abdoul-guelilou, KERE Wend-panga Georges Fabrice). Utiliser le même simulateur élimine une source de différence.
- Mettez-vous d'accord sur : mêmes paramètres communs (§3), mêmes graines (`--run=1..5`), même format de CSV (colonnes identiques) et mêmes unités.
- Échangez les CSV, tracez LEACH et DEEC sur **les mêmes graphes**. Le script `plot_results.py --compare` lit les colonnes `total_energy`, `alive` (et `nCH`, `data_sent`, `data_recv` si disponibles) : proposez à votre binôme le format du §5.3, ou convertissez ses CSV.
- Pour l'énergie normalisée, indiquez l'énergie initiale totale de DEEC : `--other-init 60` (80 nœuds à 0,5 J + 20 nœuds à 1 J).

8.2 Ce qu'il faut comparer

Métrique	Attendu pour LEACH	Attendu pour DEEC	Explication
FND	Précoce	Nettement plus tardif	DEEC élit plus souvent les nœuds ayant le plus d'énergie résiduelle par rapport à la moyenne du réseau ; LEACH peut élire un nœud presque vide.

Métrique	Attendu pour LEACH	Attendu pour DEEC	Explication
Énergie au cours du temps	Descente rapide au début	Descente plus régulière	Charge de CH mieux répartie en DEEC.
Taux de livraison	Chute plus tôt	Chute plus tardive	Les CH de LEACH meurent plus tôt ; les nœuds morts n'émettent plus.
Passage à l'échelle	Messages ≈ N + 2·nCH	Comparable, avec un peu de calcul en plus	DEEC estime l'énergie moyenne et la durée de vie du réseau ; le coût de clustering reste du même ordre.

8.3 Précautions pour comparer équitablement

- **Énergie initiale différente** : DEEC a 20 % de nœuds à 1 J, donc 60 J au total contre 50 J pour LEACH. Tracez aussi l'énergie **normalisée** (énergie résiduelle / énergie initiale) pour comparer les pentes.
- **Distance à la BS** : reliez l'écart au régime de consommation (d^2 contre d^4). La BS étant hors de la zone, la plupart des CH émettent en d^4 : un CH mal choisi coûte très cher, ce que DEEC atténue en favorisant les nœuds riches.
- **Nuance honnête** : une partie de l'avantage de DEEC vient des nœuds avancés (plus d'énergie), pas seulement de l'algorithme. Dites-le dans le rapport.

9. Rédaction du rapport (4 pages max)

Section	Taille	Contenu
1. Introduction et choix	≈ 0,3 p.	Contexte RCSF, algorithme LEACH, simulateur NS3, objectif.
2. Configuration	≈ 0,5 p.	Tableau de paramètres (§3), modèle radio, remarque sur l'unité de ε_{mp}, option de modélisation (A ou B) justifiée.
3. Résultats	≈ 1,5 p.	Courbe d'énergie + FND/HND/LND ; tableau et courbe du passage à l'échelle ; courbe du taux de livraison. Chaque figure est commentée en 2 à 3 phrases.
4. Critique de NS3	≈ 0,7 p.	Facilité d'implémentation (pas de LEACH natif, nécessité d'écrire son modèle d'énergie, courbe d'apprentissage C++) ; réalisme de la couche radio (modèles de propagation, MAC, collisions, mais écart avec le modèle de premier ordre imposé) ; performances (300 nœuds OK) ; comparaison avec ce qu'auraient apporté OMNeT++ et TOSSIM.
5. Comparaison avec DEEC	≈ 0,7 p.	Courbes superposées, tableau FND/HND/LND, explication de l'écart par les mécanismes des algorithmes (§8).
6. Conclusion	≈ 0,3 p.	Le tableau comparatif du cours est-il validé ? LEACH est-il bien un « piège » ? Limites de votre étude.

Conseils de rédaction

- Privilégiez des figures claires (2 ou 3 par page, deux par ligne si possible) plutôt que du texte.
- Chaque affirmation chiffrée doit renvoyer à une courbe ou un tableau.
- Indiquez toujours le nombre de graines et si les valeurs sont des moyennes.
- Mentionnez les hypothèses non imposées (taille de paquet, p, portée de l'ADV, repli sans CH, perte aléatoire).
- Relisez la consigne : 4 pages maximum, courbes + critique du simulateur + comparaison croisée.

10. Calendrier, pièges et check-list

10.1 Calendrier suggéré

Phase	Durée	Livrable de la phase
Mac, Vagrant, VM et NS3 installés (Étapes 0 à 3)	0,5 jour	`leach` s'exécute dans la VM
Lecture du code, validation (10 nœuds, calcul à la main), adaptation éventuelle	1 jour	CSV cohérent avec la référence Python
Expériences A, B, C (5 graines) : run_all.sh	0,5 jour	Fichiers CSV dans `results/`
Courbes et comparaison avec DEEC	0,5 jour	Figures finales
Rédaction du rapport	0,5 jour	PDF de 4 pages

10.2 Pièges à éviter

- Compter les paquets de contrôle dans le taux de livraison.
- Ajouter des nœuds avancés à LEACH (c'est réservé à SEP et DEEC).
- Garder ε_{mp} en nJ sans le discuter : d_0 serait absurde (≈ 2,8 m).
- Oublier de gérer le cas « aucun CH élu » : le programme doit définir un repli.
- Une seule graine : les courbes ne sont pas représentatives.
- Ne pas retirer les nœuds morts (ils continueraient à émettre avec une énergie négative).
- Changer des paramètres entre scénarios (taille de paquet, p) sans le dire.

- Oublier `--provider=qemu` au `vagrant up` (Vagrant cherche alors VirtualBox).
- Faire `vagrant destroy` avant d'avoir copié `results/` et `figures/` sur le Mac.
- Conclure que LEACH « livre presque tout » parce que le taux de livraison reste à 99 % : regardez aussi les données livrées par round (§6.3).
- Dépasser 4 pages ou mettre des figures sans axes ni unités.

10.3 Check-list finale

☐	Élément
☐	NS3 compile et l'exemple de base tourne
☐	Le test à 10 nœuds est validé à la main (membre et CH)
☐	Les valeurs radio sont celles du sujet (avec justification de l'unité de ε_{mp})
☐	5 graines au moins par scénario, CSV sauvegardés
☐	Courbe d'énergie, FND/HND/LND, histogramme du nombre de CH
☐	Tableau et courbe du passage à l'échelle (50, 100, 300)
☐	Taux de livraison sans paquets de contrôle
☐	Paramètres alignés avec le binôme DEEC, courbes superposées
☐	Critique de NS3 rédigée (facilité, réalisme radio, limites)
☐	Rapport de 4 pages max, figures lisibles, sources/références citées

10.4 Dépannage

Symptôme	Cause probable et solution
`vagrant up` cherche VirtualBox / « provider not found »	Ajouter `--provider=qemu`. Vérifier `vagrant plugin list` (vagrant-qemu).
Erreur sur le firmware / QEMU introuvable	Décommenter la ligne `qe.qemu_dir = "/opt/homebrew/share/qemu"` du Vagrantfile ; vérifier `brew install qemu`.
Le port SSH est déjà utilisé	Changer `qe.ssh_port = "50022"` (par exemple 50023), puis `vagrant reload`.
La VM démarre mais « timed out waiting for SSH »	Attendre (la première fois, le démarrage est lent) ; augmenter `config.vm.boot_timeout` ; relancer `vagrant up --provider=qemu`.
La compilation de NS3 est tuée (« Killed »)	Pas assez de mémoire : `VM_MEMORY=6G vagrant reload` puis `vagrant provision`.
« No space left on device » pendant la compilation	Le disque de la boîte est trop petit : libérer de la place (`df -h /`), supprimer `~/ns3/cmake-cache` inutile, ou utiliser les modules minimaux (défaut).
`./ns3 run` ne trouve pas scratch/leach	Vérifier que `~/ns3/scratch/leach.cc` existe ; relancer `run_all.sh` (il recopie et recompile).
« Impossible d'ouvrir results/… »	Le dossier du CSV n'existe pas : `mkdir -p ~/tp_rcsf/results`.
`plot_results.py` : « Aucun fichier … »	Lancer d'abord `run_all.sh`, ou passer `--results` avec le bon dossier.
Résultats très différents de la référence	Vérifier l'unité de ε_{mp}, la taille des paquets, le coût de réception du CH, le repli sans CH et la remise à zéro de G à chaque époque (§7.3).

10.5 Glossaire

Terme	Signification
RCSF / WSN	Réseau de capteurs sans fil / Wireless Sensor Network
BS / sink	Station de base qui collecte les données
CH	Cluster Head, chef de cluster
Round	Cycle complet : setup + régime permanent
Époque	1/p rounds : durée sur laquelle chaque nœud doit être CH une fois
FND / HND / LND	First / Half / Last Node Dies : rounds où le premier, la moitié, le dernier nœud meurent
TDMA	Accès au canal par créneaux temporels, évite les collisions
Agrégation	Fusion de plusieurs paquets de données en un seul
DEEC	Distributed Energy-Efficient Clustering : élection des CH pondérée par l'énergie résiduelle / moyenne, pour réseaux hétérogènes
Événements discrets	Mode de simulation où le temps saute d'un événement planifié au suivant

Références

Heinzelman, W. R., Chandrakasan, A., Balakrishnan, H. « Energy-efficient communication protocol for wireless microsensor networks », HICSS, 2000 (LEACH et modèle radio de premier ordre). · Heinzelman et al. « An application-specific protocol architecture for wireless microsensor networks », IEEE Trans. Wireless Commun., 2002. · Qing, L., Zhu, Q., Wang, M. « Design of a distributed energy-efficient clustering algorithm for heterogeneous wireless sensor networks » (DEEC), Computer Communications, 2006. · Documentation NS3 : nsnam.org (manuel et tutoriel).